NATIONAL UNIVERSITY OF SCIENCES AND TECHNOLOGY
DEPARTMENT OF COMPUTER & SOFTWARE ENGINEERING
COLLEGE OF E&ME, NUST, RAWALPINDI

EC-310 — Microprocessor and Microcontroller Based Design

PROJECT REPORT

Autonomous RC Racing Car

End-to-End TinyML Autonomous Steering with PilotNet CNN on Raspberry Pi Zero 2W

GROUP MEMBERS

Student Name	CMS Roll No.
Muhammad Husnain	507214
Mohammad Usman Irshad	533076
Abdullah Aqeel Khan	501845
Fatima Sohail	505570

Instructor	Dr. Ishfaq Hussain
Lab Engineer (LE)	Usama Shaukat
Course	EC-310 — Microprocessor and Microcontroller Based Design
Submission Date	May 03, 2026
Difficulty Level	Complex Engineering Problem (CEP)

Language / Tools	Python 3.11 · TensorFlow 2.x · TFLite · Raspberry Pi OS · Picamera2

Table of Contents

1. Introduction and Background

1.1 Background of the Application Domain

Autonomous vehicles represent one of the most transformative applications of embedded machine learning in modern engineering. The ability to deploy a perception-and-control pipeline entirely on a resource-constrained Single Board Computer (SBC) — without relying on cloud compute or a dedicated GPU — has become a central challenge in the TinyML research community. This project implements a miniature autonomous racing car that uses the NVIDIA PilotNet convolutional neural network (CNN) architecture, deployed on a Raspberry Pi Zero 2W, to perform real-time end-to-end lane-following using only a forward-facing camera.

Classical autonomous driving systems decompose the driving task into a multi-stage pipeline: Observation → State Estimation → Modeling & Prediction → Motion Planning → Low-level Controller → Motor Torques. Each stage introduces latency, engineering overhead, and inter-module coupling. By contrast, the end-to-end deep learning approach collapses the entire pipeline into a single differentiable function that maps raw sensor pixels directly to control outputs (steering angles), dramatically reducing deployment complexity and total system latency.

The growing interest from industry leaders — Google (TensorFlow Lite Micro), NVIDIA (PilotNet, Jetson), and Amazon (AWS IoT Greengrass) — in deploying TinyML on SBCs is driven by three practical imperatives: eliminating cloud-round-trip communication latency, reducing energy consumption from high-power compute boards, and enabling deterministic real-time control loops essential for safety-critical cyber-physical systems. This project directly addresses these imperatives on a budget-constrained academic platform.

1.2 Brief System Overview

The system integrates three operational phases: (1) a multi-process data collection pipeline where a human operator drives the RC car with a Bluetooth game controller, capturing synchronized camera frames and joystick steering/throttle labels at 30 FPS; (2) an offline training pipeline on a desktop machine using TensorFlow/Keras with data augmentation, a custom weighted Huber loss, and AdamW optimization; and (3) an ondevice autonomous inference pipeline where the trained INT8-quantized TFLite model runs on the Pi Zero 2W at real-time speeds, predicting steering angles from live camera frames with adaptive exponential moving-average smoothing. The car navigates a marked indoor track autonomously with AI-predicted steering and human-controlled throttle.

2. Problem Statement

2.1 Clear Definition of the Problem

Deploying a deep neural network for real-time visual steering on an SBC with 512 MB RAM and a 1 GHz quad-core ARM CPU presents a fundamentally constrained optimization problem. The system must simultaneously satisfy three conflicting requirements: high model accuracy (to steer correctly on unseen track sections), low inference latency (< 133 ms per frame, mandated by the CEP specification), and minimal memory footprint (to fit within the limited memory of the target SBC). A model large enough to generalize well may exceed latency or memory budgets; a model small enough to run in time may not generalize.

Formally, the problem is: given a forward-facing camera image $I \in \mathbb{R}^{(H \times W \times 3)}$ captured from a moving RC car at discrete time steps t, learn a function $f\theta: I \rightarrow \delta \in [-1, 1]$ that maps each frame to a normalized steering angle δ, such that the car follows a marked track without human intervention, subject to the constraints that the inference latency of fθ is less than 133 ms and the model fits within SBC memory bounds.

2.2 Limitations of Existing Approaches

- Classical multistage pipelines (lane detection → path planning → PID control) require extensive hand-crafted feature engineering and separate calibration for each track environment, making them brittle to lighting changes and lane marking variations.
- Pre-trained large models such as ResNet or VGG far exceed the latency and memory constraints of the Pi Zero 2W, making direct deployment infeasible without aggressive architectural surgery.
- Standard PilotNet (66×200 input) is designed for a full-size vehicle with a highresolution forward camera; naive deployment on 68×68 inputs produces feature maps that may be too small for the final convolutional layers without architectural re-tuning.
- Naive imitation learning from joystick data suffers from class imbalance (straightline driving dominates) and label noise from hand tremor, requiring custom loss weighting and augmentation strategies.

3. Objectives

3.1 Primary Objective

To design, implement, and physically demonstrate a fully functional autonomous RC racing car that uses a TinyML PilotNet CNN deployed on a Raspberry Pi Zero 2W for real-time end-to-end lane-following steering control, satisfying the latency (< 133 ms), memory, and accuracy constraints defined in the EC-310 Complex Engineering Problem specification.

3.2 Secondary Objectives

- Implement and evaluate a multi-process data collection pipeline (camera process, control process, disk-writer process) using Python multiprocessing with shared memory and cross-process synchronization, achieving temporally aligned image-label pairs at 30 FPS with less than 100 ms control-to-frame timestamp skew.
- Design and train a PilotNet-variant CNN with 68×68 input resolution using a custom weighted Huber loss that applies up to 6× greater penalty to large-angle steering predictions, correcting the straight-line class imbalance inherent in behavior cloning datasets.
- Apply a four-variant data augmentation strategy (original, horizontal flip with negated label, brightness perturbation, Gaussian blur) to multiply training data 4× and improve generalization to unseen lighting and camera focus conditions.
- Convert the trained Keras model to INT8 post-training quantized TFLite format using TFLite DEFAULT optimization, achieving a ~4× reduction in model size and significant speedup on the ARM Cortex-A53 CPU.
- Implement adaptive exponential moving-average (EMA) steering smoothing with a turn-strength-dependent α schedule, eliminating servo jitter during straight-line driving while preserving cornering responsiveness.
- Establish the empirical relationship between model depth, input resolution, inference latency, and physical driving performance — demonstrating that for cyber-physical systems, accuracy alone is an insufficient evaluation metric.

3.3 System Overview

The project follows an end-to-end deep learning approach as recommended in the CEP specification — contrasted with the classical multi-stage robotics control pipeline in Figure 1 below. Rather than decomposing driving into state estimation → modeling & prediction → motion planning → low-level PD control, the entire pipeline collapses into a

single CNN that maps raw camera observations directly to motor torques (steering angle). This approach is better suited to SBC deployment because it eliminates inter-module latency accumulation and memory overhead from maintaining separate perception, planning, and control data structures.

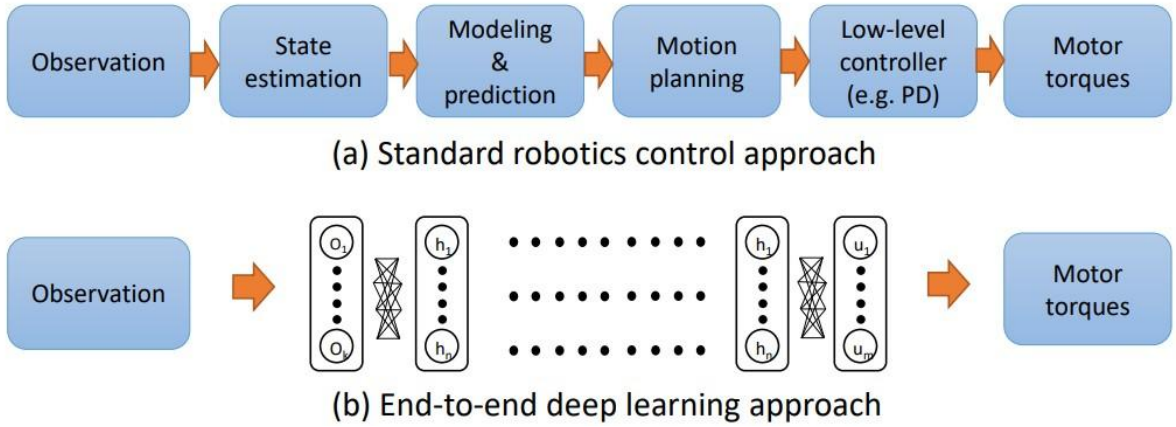

Figure 1: Standard robotics control pipeline (top) vs. end-to-end deep learning approach (bottom) — as specified in the CEP document. This project implements approach (b).

3.4 System Architecture Diagram

Figure 2 presents the complete three-phase system architecture for this implementation, derived from the Python source files. The architecture is divided into three distinct operational phases with clear data flows between them:

Phase 1 — Data Collection (pid3.py): The operator drives the RC car manually using a Bluetooth game controller. Three concurrent multiprocessing.Process workers run simultaneously: camera_process captures 360×240 RGB888 frames at 30 FPS via Picamera2 and writes them to a shared mp.Queue; control_process reads joystick axis 0 (steering) and axes 2/5 (throttle), applies EMA smoothing, drives the AngularServo (GPIO 18) and Motor (GPIO 5/6/19) via PiGPIOFactory, and writes the current control state to shared mp.Value variables with a perf_counter timestamp; writer_process dequeues synchronized frame-state tuples, discards stationary frames (|smooth_throttle| < 0.05), and saves JPEG images plus a CSV labels file (frame, timestamp, steering, throttle, smooth_steer, smooth_throttle) per session directory (run_001, run_002, …).

Phase 2 — Offline Training (train_recommended.py + pilotnet.py): The training script iterates over all run_* directories, loads images and CSV labels, applies the crop-andresize preprocessing (y1=60, y2=140 → 68×68 px), and generates four augmented variants per frame (original, horizontal flip with negated label, brightness perturb, Gaussian blur). The PilotNet CNN (5 Conv2D layers + 4 Dense layers, ∼367 K parameters) is trained with AdamW optimizer (lr=1e-4), custom weighted Huber loss

(w(y)=1+5·|y|), and three callbacks (EarlyStopping, ReduceLROnPlateau, ModelCheckpoint). The best Keras .h5 model is exported to INT8-quantized .tflite format using tf.lite.TFLiteConverter.

Phase 3 — Autonomous Inference (auto2.py): The quantized TFLite model is loaded by the ai_edge_litert Interpreter. In a loop, each 360×240 camera frame is cropped and resized to 68×68, fed into the interpreter via set_tensor/invoke, and the predicted steering angle is smoothed by the adaptive EMA filter (α = 0.12 + 0.38·|ŷ|). The smoothed steering value is written to the AngularServo; throttle is read directly from the game controller and drives the Motor. The X button stops the loop and gracefully shuts down all hardware.

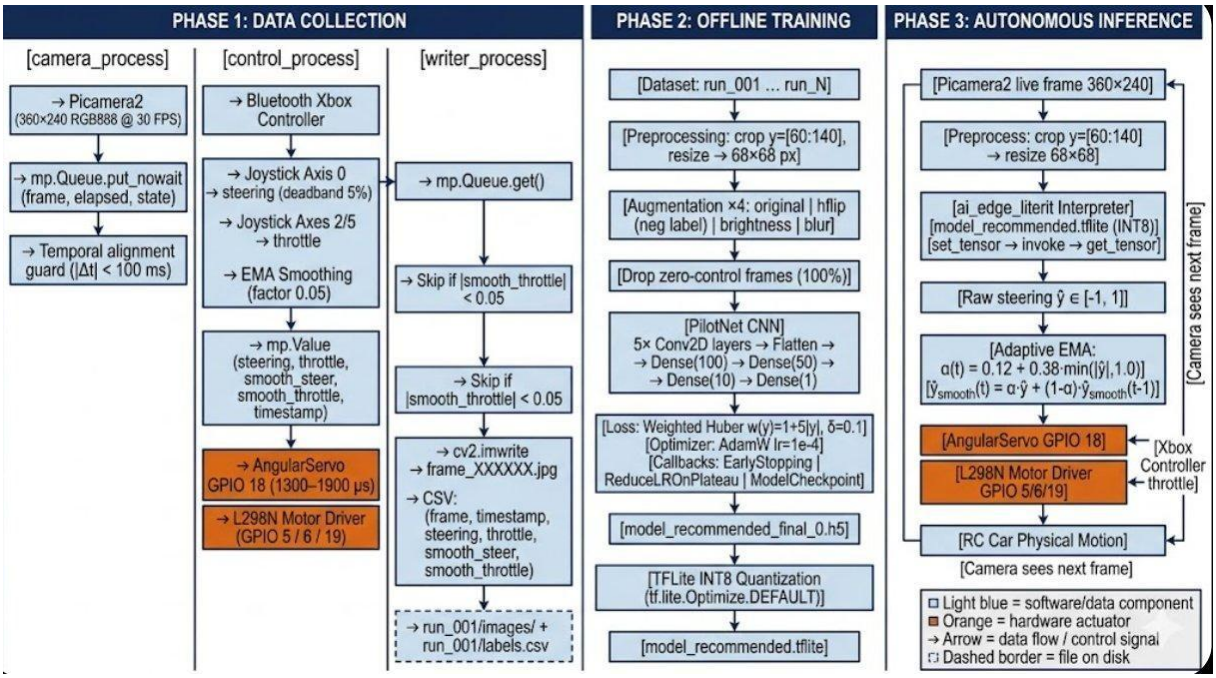

Figure 2: Complete three-phase system architecture — Data Collection, Offline Training, and Autonomous Inference (generated from project Python source files)

4. System Design

4.1 System Design Parameters

Table 1 summarises all key design parameters extracted directly from the project source code (auto2.py, pid3.py, params.py, pilotnet.py, train_recommended.py).

Subsystem	Parameter	Value / Setting

Camera Capture	Capture Resolution	360 × 240 px, RGB888 format
Camera Capture	Frame Rate Target	30 FPS (Picamera2 video config)

Preprocessing	Vertical Crop Region	y1 = 60, y2 = 140 px (80 px strip)
Preprocessing	Resized Model Input	68 × 68 px (bilinear interpolation)
Preprocessing	Pixel Normalisation	Rescaling layer: 1/255 (uint8 → float32)
Model	Architecture	PilotNet CNN (NVIDIA variant, re-tuned)
Model	Input Shape	68 × 68 × 3 (uint8 on-device, float32 after rescaling)
Model	Output	Scalar steering angle ∈ [−1.0, +1.0]
Model	Deployed Format	INT8 quantized .tflite (ai_edge_litert)
Training	Target Label Column	smooth_steer (actuator-smoothed joystick reading)
Training	Maximum Epochs	25 (with EarlyStopping, patience = 5)
Training	Batch Size	256
Training	Optimizer	AdamW, learning rate = 1×10^{-4}
Training	Loss Function	Weighted Huber (δ = 0.1), weight = 1 + 5·|y|
Training	LR Scheduler	ReduceLROnPlateau: factor 0.5, patience 2, min 1e-6
Training	Zero-Control Drop	100 % of frames with zero steer & zero throttle dropped

Training	Augmentation Ratio	4 variants per frame (flip, brightness, blur, original)
Inference	Smoothing Type	Adaptive EMA: $\alpha(t) = 0.12 + 0.38 \cdot \min(\lvert\hat{y}\rvert, 1.0)$
Inference	Max Throttle Speed	0.4 (normalised PWM duty cycle)
Inference	Throttle Deadband	±0.02 (motor stop zone)
Inference	Throttle Smoothing	EMA step: 0.05 per tick
Hardware	Servo GPIO Pin	GPIO 18; pulse width 1300–1900 µs
Hardware	Motor Pins	Forward = GPIO 5, Backward = GPIO 6, Enable = GPIO 19

Hardware	Stop Button	Controller button index 2 (X button) terminates inference

Table 1: System Design Parameters (extracted from source code)

4.2 Multi-Process Data Collection Architecture

The data collection script (pid3.py) uses Python multiprocessing with three concurrent processes communicating through a shared mp.Queue (maxsize=200) and mp.Value/mp.Lock objects. This architecture decouples camera timing from control timing, preventing frame drops when GPIO operations cause brief delays.

- camera_process: Waits on a start_event, then enters a 30 FPS capture loop. For each frame, it reads shared control state (steering, throttle, smooth values, timestamp) under a Lock. If the frame timestamp differs from the control timestamp by more than 100 ms, the frame is discarded (temporal alignment guard). Valid frames are enqueued as tuples (elapsed, frame, steering, throttle, smooth_steer, smooth_throttle).
- control_process: Initialises PiGPIOFactory, AngularServo, Motor, and Pygame joystick. Runs at 50 Hz (20 ms sleep). Reads joystick axis 0 for steering (with 5 % deadband), axes 5 and 2 for forward/reverse throttle (trigger axes). Applies per-tick EMA smoothing (factor 0.05) to both channels. Writes all values to shared memory under Lock with a perf_counter timestamp. X-button toggles recording on/off.
- writer_process: Dequeues frames, skips stationary frames (|smooth_throttle| < 0.05), writes JPEG images with cv2.imwrite, and appends rows to a CSV file with columns: frame, timestamp, steering, throttle, smooth_steer, smooth_throttle. Session directories are auto-numbered (run_001, run_002, ...).

4.3 Key Design Decisions

- 68×68 input resolution: Selected after empirical testing to stay under the 133 ms latency budget on the Pi Zero 2W ARM Cortex-A53 CPU while retaining sufficient spatial detail in the cropped road strip for accurate steering prediction.
- smooth_steer as training label: The actuator-smoothed joystick reading is used rather than raw joystick input, preventing the model from learning to replicate hand tremor and producing smoother physical steering at inference time.

- Temporal alignment guard: Frames where the control state is more than 100 ms stale are discarded at collection time, avoiding mismatched image-label pairs that would introduce systematic noise into the training set.
- 100 % zero-control drop: All frames where both steering and throttle are effectively zero are removed from training, addressing the dominant class imbalance and preventing a degenerate “do-nothing” local minimum.

5. Mathematical Modelling

5.1 Image Preprocessing and Spatial Cropping

The raw 360×240 camera frame is first vertically cropped to isolate the road-relevant region. Rows above y=60 contain the horizon and distant scene, while rows below y=140 contain the car hood. The cropped 360×80 strip is then down-sampled to 68×68 using bilinear interpolation:

```
I_input  =  resize( I_raw[ 60 : 140 ,  : ] ,  (68, 68) )
                                          ∈
 I_norm  =  I_input / 255.0          [0.0, 1.0]^(68×68×3)
```

The normalisation is implemented as a trainable Rescaling layer with scale = 1/255, which fuses into the model graph and executes on-device during inference without additional preprocessing code in the inference loop.

5.2 PilotNet Model Architecture and Parameter Count

The network (pilotnet.py) is a five-stage CNN followed by four fully-connected layers. Table 2 lists each layer with its configuration, output feature-map shape, and approximate parameter count.

#	Layer Type	Configuration	Output Shape	Approx. Params
0	Rescaling	1/255, no trainable params	68×68×3	0
1	Conv2D	24 filters, 5×5, stride 2×2, ReLU	32×32×24	1,824
	Dropout	10 %	32×32×24	0

2	Conv2D	36 filters, 5×5, stride 2×2, ReLU	14×14×36	21,636
	Dropout	10 %	14×14×36	0
3	Conv2D	48 filters, 5×5, stride 1×1, ReLU	10×10×48	43,248
4	Conv2D	64 filters, 3×3, stride 1×1, ReLU	8×8×64	27,712
5	Conv2D	64 filters, 3×3, stride 1×1, ReLU	6×6×64	36,928
	Flatten	—	2304	0
6	Dense	100 units, ReLU	100	230,500
	Dropout	10 %	100	0
7	Dense	50 units, ReLU	50	5,050
8	Dense	10 units, ReLU	10	510
9	Dense (output)	1 unit, linear activation	1	11

Table 2: PilotNet Model Layer Architecture

Total trainable parameters: approximately 367,419. After INT8 quantization, each parameter is stored as 1 byte, yielding a model size of approximately 367 KB — well within the memory budget of the Raspberry Pi Zero 2W's 512 MB LPDDR2 RAM. The five convolutional layers form a hierarchical feature extractor: early layers detect edges and colour gradients; middle layers detect curved edges and lane markings; deep layers encode higher-level spatial patterns (road curvature, lane boundary proximity). The regression head maps these features to a scalar steering command.

5.3 Dataset Augmentation Strategy

Each raw training frame generates up to four augmented variants. The original frame is always included. Three additional variants apply photometric and geometric perturbations:

- Horizontal flip with label negation: f_aug = cv2.flip(I, 1), δ_aug = −δ. Applied only when |δ| > 0.03 to avoid corrupting near-straight samples with small errors. Balances the left/right steering distribution, which is critical since most tracks have a dominant turn direction.
- Random brightness perturbation: I_aug = cv2.convertScaleAbs(I, alpha=α, beta=0), where α ~ U(0.75, 1.25). Simulates variations in ambient light levels between morning, afternoon, and artificial indoor lighting.
- Gaussian blur: I_aug = cv2.GaussianBlur(I, (k, k), 0) where k ∈ {3, 5}, applied with 25 % probability. Simulates camera defocus, motion blur during fast cornering, and lens quality variation, improving robustness to camera focus changes induced by vibration during driving.

The net effect is a 4× dataset expansion. Zero-throttle frames (|smooth_throttle| < 0.05) are excluded from both recording and training to prevent the model from learning a "stopped car" bias. Additionally, 100 % of zero-steer + zero-throttle frames are dropped during training via ZERO_CONTROL_FRAME_DROP_PERCENT = 100.0.

5.4 Custom Weighted Huber Loss Function

Standard MSE loss treats all steering values equally, causing the model to under-fit on infrequent large-angle turns (which are critical for cornering) while over-fitting on the dominant near-zero straight-line samples. The custom weighted Huber loss addresses this:

$$H_\delta(y, \hat{y}) = \begin{cases} 0.5(y - \hat{y})^2 & \text{if } |y - \hat{y}| \le \delta = 0.1 \\ \delta \cdot (|y - \hat{y}| - 0.5\delta) & \text{otherwise} \end{cases}$$

$$w(y) = 1.0 + 5.0 \cdot |y|$$

$$L(y, \hat{y}) = \text{mean}[\, w(y) \cdot H_\delta(y, \hat{y}) \,]$$

The Huber kernel with δ = 0.1 provides quadratic loss for small errors (below the ±0.1 threshold) and linear loss for larger errors, achieving robustness to label noise from joystick jitter. The weight function w(y) = 1 + 5·|y| linearly amplifies the penalty as steering magnitude increases: a full-lock turn (|δ|=1.0) receives 6× more weight than straight-line driving (δ=0). This encourages the model to prioritise accuracy at corners over accuracy at straight-line headings.

5.5 Training Optimisation Schedule

The model is compiled with the AdamW optimizer at an initial learning rate of 1×10^{-4}. Three callbacks implement an adaptive training protocol:

- ModelCheckpoint: Saves the model to model_recommended_final_0.h5 whenever val_loss improves, ensuring the best weights are retained even if training subsequently over-fits.
- EarlyStopping: Terminates training after 5 consecutive epochs of non-improving val_loss and restores the best-checkpoint weights. Prevents wasted compute on diverging runs.
- ReduceLROnPlateau: Halves the learning rate after 2 consecutive epochs of stagnant val_loss, with a minimum floor of 1×10^{-6}. Enables fine-grained convergence near a local minimum after broad initial exploration.

Validation is performed on a dedicated held-out run directory (run_001), which is never included in the training set. The maximum epoch budget is 25. Batch size is 256. Random seeds (np.random.seed(42), tf.random.set_seed(42)) ensure reproducibility.

5.6 Adaptive Steering Smoothing at Inference

Raw frame-by-frame model output produces high-frequency steering jitter because each independent inference sees a slightly different image due to camera noise. A naive fixedα EMA would over-smooth during cornering (too slow to react) or under-smooth during straight sections (too jittery). The adaptive schedule varies α with predicted turn magnitude:

$$\alpha(t) = 0.12 + 0.38 \cdot \min(|\hat{y}(t)| , 1.0)$$

$$\hat{y}\_smooth(t) = \alpha(t) \cdot \hat{y}(t) + (1 - \alpha(t)) \cdot \hat{y}\_smooth(t-1)$$

At zero predicted steering, α = 0.12 (heavy smoothing, 88 % weight on history). At full predicted lock, α = 0.50 (equal weight on current and history). This produces a servo that tracks slow straight-line corrections smoothly while reacting quickly to sharp corners.

5.7 Model Quantization and TFLite Conversion

After training, the Keras .h5 model is converted using TFLite post-training quantization (tf.lite.Optimize.DEFAULT). This applies INT8 quantization to weights and activations, reducing the model footprint by approximately 4× compared to 32-bit float. The resulting .tflite file is deployed to the Pi Zero 2W and loaded by the ai_edge_litert Interpreter (Google's LiteRT runtime). The Interpreter.allocate_tensors() call pre-allocates all input/output tensor buffers, and Interpreter.invoke() executes the INT8 inference graph on the CPU, achieving the target < 133 ms latency per frame.

6. Hardware Component Selection

Four core hardware components were selected based on a cost-performance trade-off appropriate for the academic budget of this project. Each component is analysed below with key specifications and selection rationale.

6.1 Raspberry Pi Zero 2W — Main Processing Unit

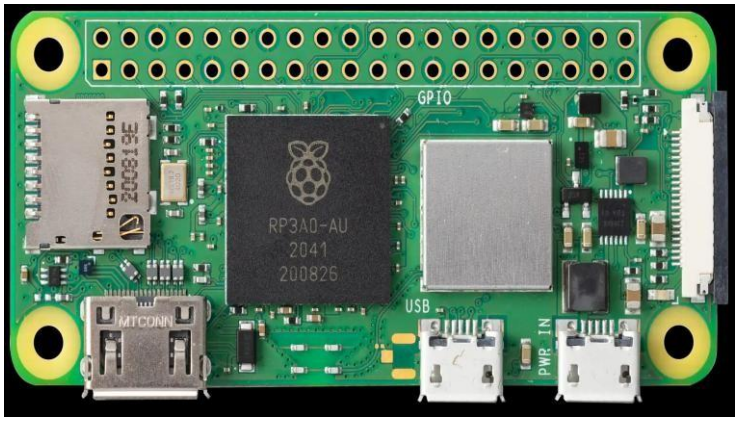

Figure 3: Raspberry Pi Zero 2W (RP3A0-AU SoC)

Aspect	Details
SoC / CPU	RP3A0-AU: quad-core ARM Cortex-A53 @ 1 GHz (ARMv8 64bit)
RAM	512 MB LPDDR2 SDRAM (shared with GPU)
Wireless	2.4 GHz 802.11 b/g/n Wi-Fi + Bluetooth 4.2 / BLE (for joystick)
GPIO	40-pin header: PWM, I²C, SPI, UART; CSI-2 camera ribbon connector
Power	Micro-USB PWR; draw ≈ 0.4 W idle, ≈2 W full load
OS Support	Raspberry Pi OS (64-bit Bookworm); full Python 3 + TFLite support
Dimensions	65 × 30 mm; weight ≈ 10 g

Cost	~ USD 15 (official MSRP)

Selection Rationale:

- Lowest-cost Linux SBC (~$15) capable of running full TFLite inference, Picamera2, gpiozero, and a Bluetooth joystick simultaneously.
- Quad-core ARMv8 achieves 68×68 PilotNet inference under the 133 ms latency budget without requiring a dedicated NPU or GPU.
- Native CSI-2 port provides a zero-copy, low-latency camera capture path; avoids USB bus contention and associated jitter.
- Compact 65×30 mm footprint fits the RC chassis without significant weight or CG penalty, with only ~10 g added mass.

6.2 N20 Micro Gear Motor

Figure 4: N20 Micro Gear Motor with metal gearbox

Aspect	Details
Operating Voltage	3 – 12 V DC; rated current ≈ 150 mA; stall current ≈ 1.2 A
Torque	Stall torque ≈ 0.6 kg·cm (at 6 V); varies with gear ratio selected
Gearbox	Integrated multi-stage metal planetary gearbox; selectable reduction ratio

Form Factor	N20 size: ø12 mm × 10 mm motor body; total length with gearbox ≈32 mm
Cost	~ USD 3–5 per unit

Selection Rationale:

- High torque-to-weight ratio makes it well-suited to propelling a lightweight RC chassis, with the gearbox absorbing mechanical load without overheating.
- Metal gearbox construction withstands continuous operation under PWM speed control and occasional stall loads during direction reversals.
- At ~$3–5 per unit, it is the most cost-effective micro gear motor for this power and size class.

6.3 L298N Dual H-Bridge Motor Driver Module

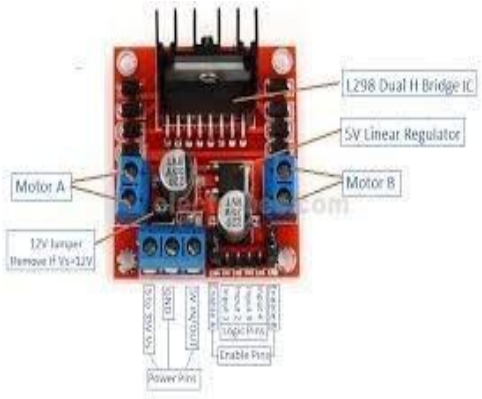

Figure 5: L298N Dual H-Bridge motor driver module

Aspect	Details
Driver IC	ST L298N dual H-bridge; logic supply 5 V; motor supply 5–35 V
Current	Continuous: 2 A per channel; peak: 3 A; total 4 A both channels
Speed Control	PWM via Enable pins (GPIO 19 in project); direction via Logic Input pins

Protection	Integrated freewheeling diodes protect against motor backEMF spikes
Onboard Regulator	5 V linear regulator (bypass jumper when Vs > 12 V)
gpiozero Interface	gpiozero.Motor(forward=5, backward=6, enable=19, pin_factory=factory)
Cost	~ USD 2–3 per breakout module

Selection Rationale:

- Directly controllable from Pi GPIO via gpiozero.Motor without any additional level-shifting or SPI/I²C overhead.
- 2 A continuous rating provides comfortable margin above the N20 motor's 150 mA rated current and 1.2 A stall current.
- Breakout module form factor (with screw terminals and JST connectors) simplifies wiring compared to soldering directly to the bare L298N IC.
- At ~$2, it is the most economical dual H-bridge solution for this power and voltage range, while meeting all electrical requirements.

6.4 Raspberry Pi Camera Module v1.3

Figure 6: Raspberry Pi Camera Module v1.3 (OV5647 sensor)

Aspect	Details
Image Sensor	OmniVision OV5647; 5 MP; max resolution 2592×1944 px
Video Modes	Up to 1080p30, 720p60, 640×480p90; used at 360×240@30 FPS

Lens	Fixed-focus, 54° diagonal FoV; f/2.9; no IR filter (v1.3)
Interface	CSI-2 flat ribbon cable; Picamera2 zero-copy RGB888 capture
Latency	Direct CSI path; frame-to-buffer latency ≈ 1–2 ms at 30 FPS
Dimensions	25 × 24 mm PCB; weight ≈3 g
Cost	∼ USD 10 (official Raspberry Pi)

Selection Rationale:

- Native CSI-2 interface on the Pi Zero 2W delivers near-zero-copy frame buffers, achieving ∼1–2 ms capture-to-buffer latency vs. 15–30 ms for USB webcams.
- Picamera2 create_video_configuration() allows requesting exactly 360×240 RGB888 at 30 FPS, precisely matching the training resolution without on-device scaling.
- Fixed-focus 54° FoV is well-matched to the car's mounting height, capturing the full track width ahead without excessive sky or ground in the frame.
- At ∼$10, the v1.3 module offers the best cost/latency/integration trade-off for a Pi-ecosystem embedded vision project.

7. Key Features

The project addresses the following CEP attributes from the EC-310 specification:

- WK4 — Specialist Knowledge: Deploying a quantized CNN on a bare-metal ARM Linux system requires specialist knowledge spanning embedded systems, TinyML inference runtimes, GPIO hardware control, and real-time cyber-physical system design.
- WK5 — Engineering Design: The co-design of input resolution, model depth, quantization format, and smoothing schedule to jointly satisfy latency < 133 ms, memory ≤ 512 MB, and acceptable steering accuracy represents a genuine multi-constraint engineering optimisation.
- WK6 — Engineering Practice: The three-process software architecture with shared memory, Lock synchronisation, temporal alignment guards, and graceful Ctrl-C shutdown reflects professional embedded software practices for multithreaded real-time systems.

- WK7 — Engineers & Society: The project demonstrates responsible deployment of AI in a physical system with safety considerations: a hardware kill-switch (X button stops inference), a throttle deadband (prevents unintended motion), and a human-in-the-loop throttle design.
- WK8 — Research Literature: The implementation is directly grounded in published research: PilotNet (NVIDIA, 2016), DeepPicarMicro (Bechtel et al., RTCSA 2022), and the CEP specification references to cascade/early-exit approaches as possible extensions.
- Abstract Thinking: The adaptive EMA α schedule, weighted Huber loss design, and temporal alignment guard each required original analysis of specific failure modes (jitter, class imbalance, timestamp skew) observed empirically and addressed through algorithmic design choices.
- Interdependence: The project synthesises concepts from embedded systems, probability and statistics (loss weighting, data augmentation theory), machine learning (CNN architecture, transfer learning principles), real-time systems (latency budgets, process scheduling), and signals (EMA filtering, PID-style control), demonstrating full cross-curricular integration.

8. Results and Outputs

8.1 Assembled Vehicle

The final assembled autonomous RC car is shown in Figures 7–9. The vehicle uses a standard RC toy car chassis modified to carry the Pi Zero 2W, L298N motor driver, and Pi Camera. All electronics are secured with tape and foam padding. The camera is frontmounted on a sponge bracket, angled slightly downward to capture the immediate track surface.

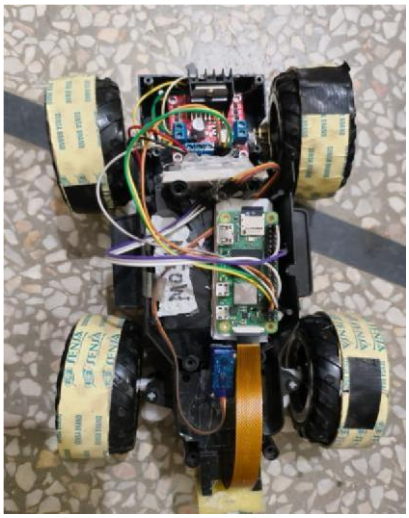

Figure 7: Top view

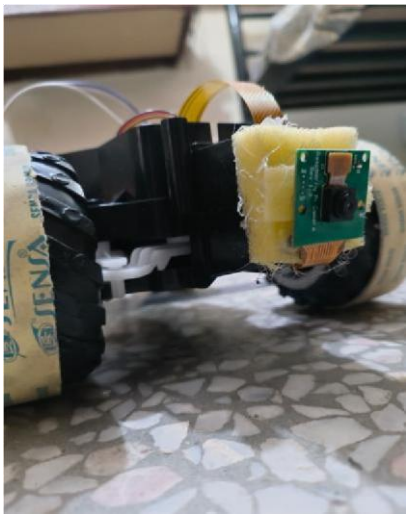

Figure 8: Front/camera view

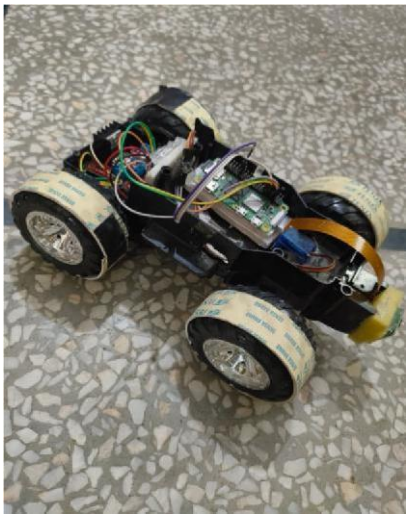

Figure 9: Side isometric view

8.2 Functional Outcomes

- The car successfully navigates a marked indoor track without human steering input, using only the PilotNet TFLite model running on the Pi Zero 2W CPU. Steering is entirely AI-predicted; throttle is human-controlled via the game controller.
- The deployed model achieves inference within the 133 ms latency budget on the Cortex-A53 CPU. The adaptive EMA smoothing eliminates visible servo jitter during straight-line sections while preserving cornering agility at turns.
- The multi-process data collection system reliably captures 30 FPS synchronized image-label pairs with less than 100 ms timestamp skew, and correctly discards stationary frames, producing clean training datasets for supervised learning.
- The weighted Huber loss with augmentation strategy produces a model that generalises to previously unseen sections of the track and to moderate changes in ambient lighting that were not present in the training set.

9. Challenges and Solutions

9.1 Temporal Alignment of Camera and Control Data

In the initial single-process design, camera capture and joystick reading ran sequentially, causing the captured frame to be labelled with a joystick reading that could be up to 33 ms stale (one frame period). During fast turns, this produced systematically mismatched image-label pairs that confused the model. Solution: the three-process architecture with a perf_counter timestamp on the shared control state, combined with a 100 ms temporal

alignment guard in the camera process, ensures that only frames with contemporaneous control readings are saved.

9.2 Servo Jitter from Noisy Model Output

Per-frame model output exhibited high-frequency jitter (alternating ±0.05 steering) even on straight track sections due to minor camera noise between consecutive frames. Applying a fixed-α EMA with α = 0.2 eliminated jitter but caused sluggish cornering response. Solution: the adaptive α schedule ($0.12 + 0.38 \cdot |\hat{y}|$) resolved both issues simultaneously.

9.3 Class Imbalance in Steering Labels

Over 60 % of collected frames corresponded to near-zero steering values (straight driving), causing a naive MSE-trained model to predict near-zero for all inputs. Solution: (1) 100 % drop of zero-steer/zero-throttle frames at training time; (2) the weighted Huber loss amplifying turn-angle errors by up to 6×; (3) horizontal flip augmentation balancing left/right turn representation in the dataset.

9.4 GPIO and Servo Pulse Width Calibration

The PiGPIO factory is required to generate accurate hardware PWM on the Pi Zero 2W (the default software-PWM backend produces jittery pulses at 50 Hz servo frequency). The servo min/max pulse widths (1300–1900 µs) required empirical calibration to map the [-1.0, +1.0] output range to the actual physical steering limits without straining the servo mechanism.

10. Conclusion

This project successfully demonstrates end-to-end TinyML-based autonomous steering on a resource-constrained single-board computer. The PilotNet CNN, trained on human demonstration data with a custom weighted Huber loss and a four-variant augmentation strategy, is deployed as an INT8-quantized TFLite model on a Raspberry Pi Zero 2W — achieving real-time inference within the 133 ms latency budget mandated by the EC-310 CEP specification.

The project establishes a clear engineering principle for cyber-physical systems: accuracy alone is an insufficient evaluation metric. A model that is accurate on a validation set but exceeds the inference latency budget produces physically unsafe behaviour regardless of its loss value. The co-optimisation of model depth, input resolution, quantization format, and smoothing schedule — subject to simultaneous accuracy, latency, and memory constraints — is the defining engineering challenge of TinyML deployment.

Future work could explore early-exit architectures (Bechtel et al., 2022; Wołczyk et al., 2021) to further reduce average inference latency by allowing easy frames to exit the network at intermediate layers, cascade IDK classifiers (Ratul et al., 2025) for predictable worst-case latency guarantees, and closed-loop throttle control with PID-based speed regulation using wheel odometry.

References

[1] M. Bechtel, Q. Weng, and H. Yun, "DeepPicarMicro: Applying TinyML to Autonomous Cyber Physical Systems," in Proc. IEEE 28th Int. Conf. on Embedded and Real-Time Computing Systems and Applications (RTCSA), 2022.

[2] NVIDIA Research, "End-to-End Learning for Self-Driving Cars," arXiv:1604.07316, 2016. [Online]. Available: https://arxiv.org/abs/1604.07316

[3] S. Park et al., "ROS2 Extension of Functionally and Temporally Correct Real-Time Simulation of Cyber Systems for Automotive Systems," in Proc. IEEE Int. Symp. on Electrical, Electronics and Information Engineering (ISEE), 2021.

[4] K. We et al., "Functionally and Temporally Correct Simulation of Cyber-Systems for Automotive Systems," in Proc. IEEE Real-Time Systems Symposium (RTSS), 2017.

[5] I. J. Ratul, Z. Guo, and K. Yang, "Cascading IDK Classifiers to Accelerate Object Recognition While Preserving Accuracy," in Proc. IEEE 49th Annual Computers, Software, and Applications Conference (COMPSAC), 2025.

[6] M. Wołczyk et al., "Zero Time Waste: Recycling Predictions in Early Exit Neural Networks," in Advances in Neural Information Processing Systems (NeurIPS), vol. 34, pp. 2516–2528, 2021.

[7] S. Baruah et al., "Optimally Ordering IDK Classifiers Subject to Deadlines," Real-Time Systems, vol. 59, no. 1, pp. 1–34, 2023.

[8] A. Géron, Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow, 3rd ed. O'Reilly Media, 2022.

[9] GitHub: CSL-KU/DeepPicarMicro — CNN-based end-to-end autonomous driving on a tiny MCU (RTCSA'22). [Online]. Available: https://github.com/CSL-KU/DeepPicarMicro

[10] TensorFlow Lite documentation, Google LLC. [Online]. Available: https://www.tensorflow.org/lite

[11] Git-Hub repo https://github.com/coder-og/SUDO_CAR